

Competitive Programming

Section 74: Geometric Sweeps And Proximity

Competitive Programming Course

Table of contents

1	Sweep events are geometric state changes	3
2	Rectangle union area	4
3	Closest pair divide and conquer	5
4	Worked trace: equal-coordinate event order	6
5	Worked trace: covered length is geometric, not a count	7
6	Worked trace: perimeter needs component count	8
7	Closest-pair packing intuition	9
8	C++ and Python implementation notes	10
9	Proof and testing checklist	11
10	Exercises	12

1 Sweep events are geometric state changes

A sweep line is useful when the answer at coordinate x depends only on objects currently crossing that line. Correctness begins with event ordering. For closed horizontal segments, process **add**, then vertical **query**, then **remove** at the same x ; this makes both endpoints active.

Compress y -coordinates and store active horizontal counts in a Fenwick tree. A vertical segment asks for the number of active y -values in its closed range. The geometry has become a one-dimensional range-sum problem.

2 Rectangle union area

Between consecutive x-events, the covered y-length is constant. Area gained is `delta_x * covered_y`. A segment tree over elementary compressed y-intervals stores a cover count and covered length. If a node's cover count is positive, its full geometric interval is covered; otherwise its length is the sum of its children. Group equal-x events so zero-width ordering cannot create area.

3 Closest pair divide and conquer

Split points by x . After recursively solving both halves, only a vertical strip of width $\sqrt{d}$ can contain a better cross-half pair. Keep each recursive result sorted by y and merge them. In the strip, compare forward only while the y -gap squared is below d . Packing equal squares shows that only constantly many later points need examination.

The important subtlety is that the recursive array must remain y -sorted while the split coordinate is saved before recursion. Duplicate points immediately produce distance zero and must not break the strip logic.

4 Worked trace: equal-coordinate event order

A horizontal segment spans x from 2 through 5, and a vertical segment lies at $x=5$. If removal is processed before the vertical query, their shared endpoint is missed. For closed segments the correct order is

```
add at x -> query at x -> remove at x.
```

Event order is part of the mathematical model. Write whether each object is active on $[start, end]$, $[start, end)$, or another convention before choosing sort keys.

5 Worked trace: covered length is geometric, not a count

After compression, consecutive y-coordinates might be 0, 2, 7. The segment- tree leaves represent intervals of lengths 2 and 5, not two unit cells. A node with positive cover count stores `ys[right]-ys[left]`; only an uncovered internal node sums its children. Counting compressed indices would produce 2 where the actual covered length is 7.

Between $x=3$ and $x=8$, if the active covered y-length is 7, the union gains area $5*7=35$. Events at $x=8$ affect the next strip, never this one.

6 Worked trace: perimeter needs component count

Suppose the active y-union is $[0, 2]$ together with $[5, 8]$. Its covered length is 5, but a horizontal sweep of width dx exposes four horizontal boundary segments, contributing

```
2 * number_of_components * dx = 4*dx.
```

Thus a perimeter tree must additionally store the number of covered components and whether its left and right endpoints are covered. When merging children, subtract one component if the left child's right boundary touches the right child's left boundary.

At one x-coordinate, vertical perimeter is the measure of the symmetric difference between coverage immediately before and after the events. Processing all additions before removals and summing changes in covered length computes that symmetric difference even when rectangles start and end together.

7 Closest-pair packing intuition

Once both recursive halves have no pair closer than $\text{sqrt}(d)$, divide the crossing strip into boxes of side $\text{sqrt}(d)/2$. A box cannot contain two points from the same half without contradicting its recursive answer. Only constantly many boxes intersect the relevant forward y-window, so each point compares with only constantly many successors. The implementation may simply continue while the y-gap squared is below d ; the proof explains why that loop is short.

8 C++ and Python implementation notes

Represent each sweep event with an explicit type or signed delta and document its tie order. Coordinate-compression maps should include both endpoints of every updated interval. Segment-tree recursion depth is logarithmic, but closest-pair recursion in Python should reuse buffers where practical to avoid quadratic slicing and allocation.

Squared distances require wide integers: coordinate differences near $2e9$ already square near the signed 64-bit boundary. Avoid square roots inside comparisons. For union area and perimeter, multiply geometric lengths only after promoting to the required type.

9 Proof and testing checklist

- What remains constant between two consecutive sweep coordinates?
- Which objects are active exactly at an endpoint?
- What does every segment-tree node represent in original coordinates?
- Which packing fact bounds the closest-pair strip loop?
- Are duplicates, zero distance, touching rectangles, and equal events handled?

10 Exercises

A focuses on inclusive event order. B combines a sweep with a geometric-length segment-tree invariant. C uses a geometric packing proof to remove a quadratic comparison loop. D is blind and asks which additional boundary information is needed when rectangle-union perimeter is requested together with area.